Introduction

Kernel SVMs extend the linear SVM framework to non-linear decision boundaries via the kernel trick: replace every inner product $x_i^\top x_j$ in the dual formulation with a kernel function $K(x_i, x_j)$ that corresponds to an inner product in a high-dimensional (often infinite-dimensional) implicit feature space. The Radial Basis Function (RBF) kernel is the workhorse, capable of representing essentially any smooth decision boundary given enough training data and appropriate hyperparameter tuning.

Prerequisites

A working understanding of linear SVMs, inner-product spaces, the dual formulation of constrained optimisation, and Mercer's theorem (the conditions a function must satisfy to be a valid kernel).

Theory

Common kernels:

- **RBF (Gaussian)**: $K(x, x') = \exp(-\gamma \|x - x'\|^2)$. Universal approximator; the default choice for most non-linear problems.
- **Polynomial**: $K(x, x') = (x^\top x' + c)^d$. Useful when polynomial structure is expected from domain knowledge.
- **Linear**: $K(x, x') = x^\top x'$. Reduces to the standard linear SVM.
- **Sigmoid**: $K(x, x') = \tanh(\alpha x^\top x' + c)$, related to single-layer perceptrons.

Two hyperparameters dominate: the cost parameter C (margin tightness, inherited from linear SVM) and a kernel-specific parameter (γ for RBF, d and c for polynomial). RBF γ trades bias for variance: small γ produces a smoother decision boundary, large γ produces a wigglier one with high variance.

Assumptions

Features are appropriately scaled; sufficient data to support the kernel's implicit dimensionality (RBF in particular grows in effective complexity with γ); C and γ are tuned jointly because they are correlated.

R Implementation

```r
library(e1071)

set.seed(2026)
n <- 300
theta <- runif(n, 0, 2 * pi)
r <- c(rep(1, n/2), rep(2.5, n/2)) + rnorm(n, 0, 0.3)
df <- data.frame(x1 = r * cos(theta), x2 = r * sin(theta),
                 y  = factor(rep(c("inner", "outer"), each = n/2)))
df[, 1:2] <- scale(df[, 1:2])

svm_rbf <- svm(y ~ ., data = df,
               kernel = "radial", cost = 1, gamma = 0.5,
               scale = FALSE)
table(predict(svm_rbf, df), df$y)

# Grid-search C x gamma via tune()
tune_out <- tune(svm, y ~ ., data = df, kernel = "radial",
                 ranges = list(cost = c(0.1, 1, 10),
                               gamma = c(0.1, 0.5, 1)))
tune_out$best.parameters
```

Output & Results

The RBF-kernel SVM separates the two concentric classes — a non-linear pattern that no linear classifier can solve. `tune()` performs grid-search cross-validation over C and γ and returns the best combination by CV accuracy.

Interpretation

A reporting sentence: "The RBF-kernel SVM correctly separated the concentric-rings classes with grid-CV-selected $C = 1$ and $\gamma = 0.5$ (CV accuracy 0.96, test accuracy 0.94); the linear-kernel baseline managed only 0.51, confirming the non-linear structure cannot be captured without the kernel trick." Always state both hyperparameters and how they were tuned.

Practical Tips

- Always standardise features; kernel distances are scale-sensitive and unscaled features bias the geometry.
- For RBF, γ trades bias (small γ, smoother boundary) for variance (large γ, wigglier boundary); tune jointly with C on a logarithmic 2D grid.
- C and γ are correlated; larger C paired with smaller γ often gives similar performance to smaller C with larger γ.
- High-degree polynomial kernels are rarely useful in practice; RBF subsumes most cases and is more numerically stable.
- For large n ($> 100{,}000$), kernel SVMs become computationally expensive ($O(n^2)$ to $O(n^3)$ training); consider Nyström approximation (`kernlab::kha`) or fall back to primal linear SVM on engineered features.
- For probability output, set `probability = TRUE` in `e1071::svm()`; it fits a Platt-scaling layer on top of the SVM decision values.

R Packages Used

`e1071::svm()` for the canonical kernel SVM with RBF, polynomial, and sigmoid options; `kernlab::ksvm()` for an alternative with richer kernel customisation; `LiblineaR` for the primal linear-only fast variant; `parsnip::svm_rbf()` and `svm_poly()` for tidymodels integration; `caret` for cross-validated joint tuning.

For Reviewers

What to look for in a paper using this method.

- **Common misapplications.**
- **Diagnostics that should be reported but often aren't.**
- **Red flags in tables and figures.**
- **What to verify.**
- **What an adequate Methods paragraph must contain.**

See also — labs in this chapter

- Cross-validation, nested CV, bootstrap .632+
- Regularisation: ridge, lasso, elastic net
- Trees, random forests, gradient boosting
- Interpretability and SHAP
- Tabular neural networks with torch
- Imaging and sequence models intro
- tidymodels pipelines
- FDR, knockoffs, replication crisis